Generador de Configuración de DAGs para Apache Airflow



Alberto Luis Vigna Arroyo

12 de septiembre del 2025

Introducción 1
Objetivo: 1
Alcance: 1
Tecnologías Utilizadas 1
Licencia 1
Vista General del Generador 1
Pantalla Principal 1
Elementos Comunes de la Interfaz 1
- Tooltips Informativos: 1
- Validaciones en Tiempo Real: 1
- Vista Previa del JSON dinámicamente: 1
Secciones de Configuración 1
Información Básica 1
- DAG ID: 1
- Propietario (Owner): 1
- Descripción: 1
- Documentación (Markdown): 1
Programación y Fechas 1
- Programación (Cron): 1
- Fecha de Inicio (Start Date): 1
- Fecha de Fin (End Date): 1
Configuración de Ejecución 1
- Max Ejecuciones Activas (max_active_runs): 1
- Concurrencia DAG (concurrency): 1
- Max Tareas Activas (max_active_tasks): 1
- Timeout DAG Run (dagrun_timeout): 1
- Vista Predeterminada (default_view): 1
- Orientación del Gráfico (orientation): 1
- Pausado al Crear (is_paused_upon_creation): 1
Notificaciones por Email 1

- Lista de Correos Electrónicos (email): 1
- Email en Fallo (email_on_failure): 1
- Email en Reintento (email_on_retry): 1
- Email en Éxito (email_on_success): 1
Reintentos y Timeouts 1
- Número de Reintentos (retries): 1
- Retraso entre Reintentos (retry_delay): 1
- Max Retraso Reintento (max_retry_delay): 1
- Timeout Ejecución (execution_timeout): 1
- Timeout General (timeout): 1
- SLA (sla): 1
- Backoff Exponencial en Reintentos (retry_exponential_backoff: 1
- Depende del Pasado (depends_on_past): 1
Configuración Avanzada 1
- Pools y Queues 1
  - Pool (pool): 1
  - Pool Slots (pool_slots): 1
  - Queue (queue): 1
- Prioridad y Peso 1
  - Peso de Prioridad (priority_weight): 1
  - Regla de Peso (weight_rule): 1
- Trigger Rules y Callbacks 1
  - Trigger Rule (trigger_rule): 1
  - Callback SLA Miss (sla_miss_callback): 1
  - Callback al Fallo (on_failure_callback): 1
  - Callback en Éxito (on_success_callback): 1
  - Callback en Reintento (on_retry_callback): 1
- Otras Configuraciones 1
  - Tags (tags): 1
  - Catchup (catchup): 1
  - Provide Context (provide_context): 1
Editor Dinámico de JSON: 1

Introducción

Objetivo:

El **Generador de Configuración de DAGs para Apache Airflow** es una aplicación desarrollada con el fin de optimizar y estandarizar el proceso de creación de flujos de trabajo dentro de entornos de orquestación de datos. Apache Airflow, como herramienta ampliamente adoptada en la ingeniería de datos, requiere de configuraciones precisas en cada DAG para garantizar la correcta ejecución de tareas en producción.

El objetivo principal de este generador es **proporcionar una interfaz amigable e intuitiva** que permita la definición de parámetros esenciales y avanzados sin necesidad de redactar código manual. De este modo, se busca:

- Reducir la complejidad en la configuración inicial de los DAGs.
- Disminuir la probabilidad de errores humanos asociados a parámetros mal definidos.
- Facilitar la adopción de buenas prácticas de orquestación en equipos multidisciplinarios.
- Ahorrar tiempo en la generación de configuraciones repetitivas o estandarizadas.

En consecuencia, esta herramienta actúa como un **asistente de configuración** que democratiza el acceso a la creación de DAGs, permitiendo que tanto perfiles técnicos como de gestión puedan interactuar con la plataforma de forma ágil.

Alcance:

El sistema ofrece una amplia gama de funcionalidades que cubren las principales necesidades de configuración de un DAG en Apache Airflow. Entre ellas se incluyen:

- **Definición de parámetros básicos** como el identificador único del DAG, propietario, descripción y documentación asociada.
- **Programación y fechas**, incluyendo soporte para expresiones cron y la definición de fechas de inicio y finalización.
- **Configuración de ejecución**, donde se pueden establecer límites de concurrencia, ejecución en paralelo y parámetros de control de flujo.
- **Gestión de notificaciones**, habilitando alertas automáticas por correo electrónico en casos de éxito, fallo o reintento.
- **Manejo de reintentos y timeouts**, proporcionando flexibilidad para escenarios de resiliencia.
- **Opciones avanzadas**, tales como asignación de pools y colas, definición de reglas de disparo (trigger rules), control de acceso mediante JSON y parámetros personalizados.
- **Edición dinámica en formato JSON**, permitiendo agregar objetos y propiedades anidadas que amplían la capacidad de personalización.
- **Vista previa en tiempo real y exportación**, con la posibilidad de copiar la configuración al portapapeles o descargarla en formato JSON listo para ser integrado en Airflow.

El alcance del sistema no pretende reemplazar la totalidad de funcionalidades que ofrece Apache Airflow ni cubrir casos específicos de ejecución compleja. Más bien, su propósito es actuar como una **herramienta de apoyo** que acelera el diseño y despliegue de DAGs, asegurando coherencia en los parámetros básicos y fomentando la estandarización organizacional.

Tecnologías Utilizadas

La arquitectura del proyecto ha sido diseñada con un enfoque ligero, modular y fácilmente desplegable en entornos de desarrollo y producción. Las tecnologías empleadas son:

- **Backend (Python)**:
  - **Flask 2.3.3**: framework web minimalista que sirve como núcleo de la aplicación.
  - **Gunicorn 21.2.0**: servidor WSGI que asegura un rendimiento óptimo en entornos de producción.
  - **Werkzeug 2.3.7**: librería fundamental para la gestión de utilidades WSGI y HTTP.
  - **Jinja2 3.1.2**: motor de plantillas que facilita la generación dinámica de interfaces HTML.
  - **MarkupSafe 2.1.3**: encargado de garantizar la seguridad en el renderizado de plantillas.
  - **itsdangerous 2.1.2**: utilizado para la gestión de firmas criptográficas en sesiones y tokens.
  - **click 8.1.7**: soporte para la creación de comandos de consola que agilizan la administración del sistema.
  - **blinker 1.6.3**: librería de señales que mejora la comunicación interna de la aplicación.
- **Frontend (UI/UX)**:
  - **HTML5, CSS3 y JavaScript** como lenguajes base de la interfaz de usuario.
  - **Bootstrap 5.3.0**, que aporta un diseño responsivo y consistente en distintos dispositivos.
  - **Font Awesome 6.4.0**, utilizado para iconografía clara y moderna que enriquece la experiencia de usuario.

Este stack tecnológico fue seleccionado por su **simplicidad, estabilidad y facilidad de integración** en pipelines de despliegue continuo, permitiendo además futuras extensiones hacia otros módulos.

Licencia

El proyecto se distribuye bajo los términos de la **Licencia MIT**, una de las licencias de software libre más ampliamente adoptadas en la industria.

Dicha licencia otorga a cualquier persona el derecho de:

- Usar, copiar y modificar el software sin restricciones.
- Distribuir versiones modificadas o no modificadas del código.
- Integrar el software en productos comerciales o de código abierto.

Las únicas condiciones establecidas son:

1. Mantener el aviso de copyright del autor original en todas las copias o partes sustanciales del software.
2. Reconocer que el software se proporciona **"tal cual"**, sin garantías de ningún tipo, ya sean expresas o implícitas.

En consecuencia, el proyecto ofrece plena libertad de uso y adaptación, promoviendo la **colaboración abierta** y el **intercambio de conocimiento** en la comunidad de ingeniería de datos.

Vista General del Generador

El **Generador de Configuración de DAGs para Apache Airflow** presenta una interfaz gráfica clara y organizada, diseñada para guiar al usuario paso a paso en la definición de los parámetros de un DAG. La aplicación se divide en secciones temáticas que agrupan los distintos aspectos de la configuración, lo que facilita la navegación y asegura un flujo lógico en la construcción de la configuración final.

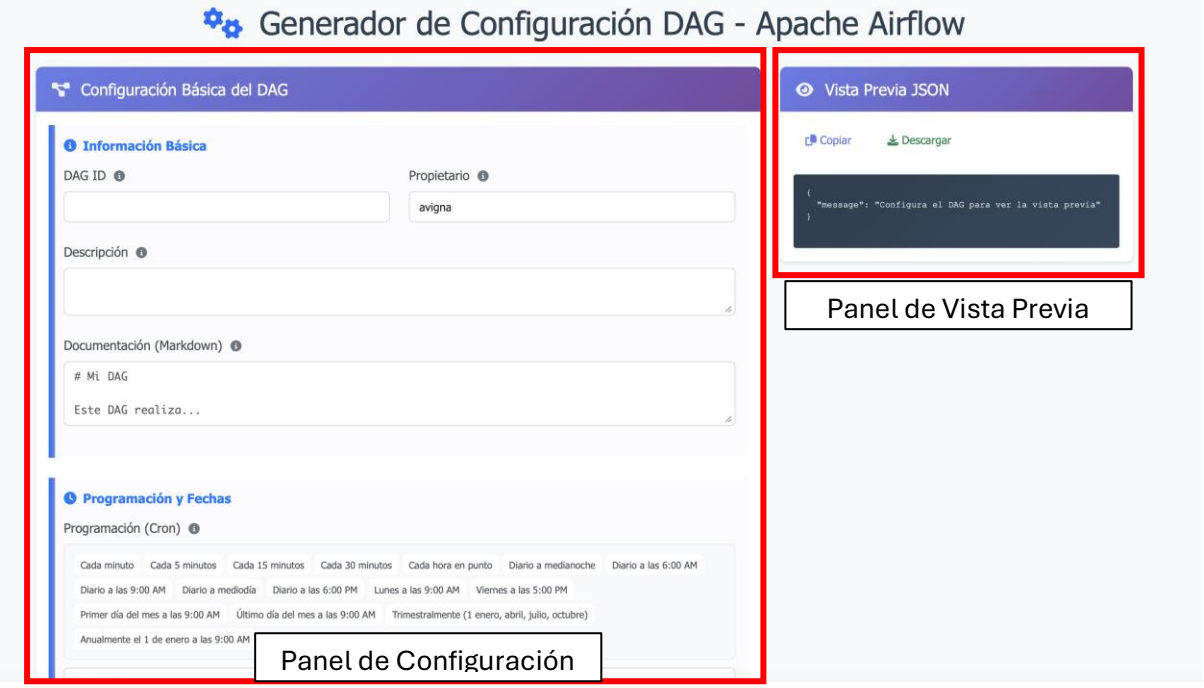
El flujo de trabajo general puede resumirse en tres etapas principales:

1. **Definición de parámetros de configuración**: el usuario completa las distintas secciones del formulario, agrupadas en bloques temáticos (información básica, programación, ejecución, etc.).
2. **Visualización en tiempo real**: el sistema genera una vista previa en formato JSON con cada cambio realizado, permitiendo validar la configuración antes de exportarla.
3. **Exportación de resultados**: una vez completada la configuración, el usuario puede copiar el JSON al portapapeles o descargarlo como archivo para integrarlo directamente en su entorno Airflow.

Pantalla Principal

La pantalla principal está dividida en dos paneles:

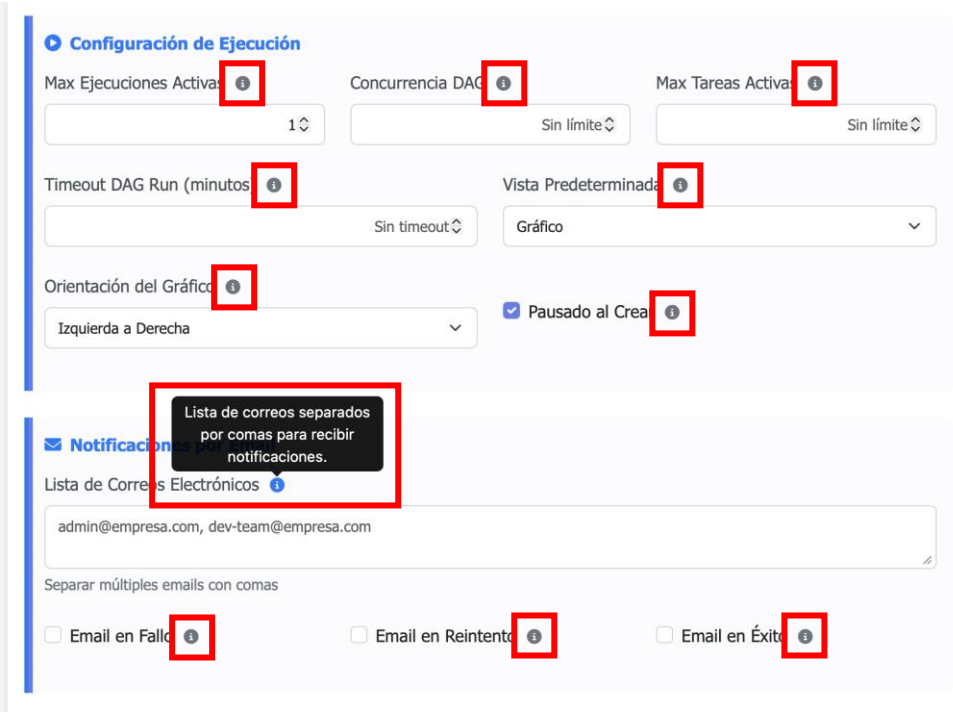
- **Panel de Configuración (lado izquierdo)**: aquí se encuentran todas las secciones del formulario, ordenadas jerárquicamente. Cada bloque contiene campos, selectores y casillas de verificación con explicaciones contextuales (tooltips).
- **Panel de Vista Previa (lado derecho)**: muestra en tiempo real el archivo JSON que se genera a partir de los datos ingresados en el formulario. Este panel cuenta con botones de acción para copiar o descargar la configuración.



Elementos Comunes de la Interfaz

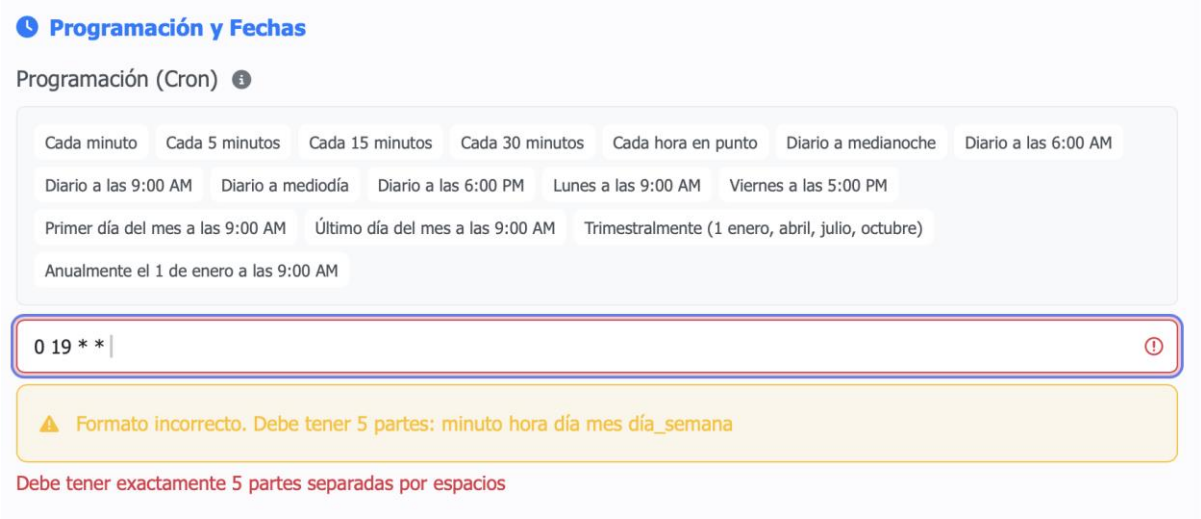
- ### Tooltips Informativos:

  Cada campo dispone de un icono de ayuda que, al pasar el cursor, muestra una breve explicación sobre su propósito. Esto facilita que incluso usuarios sin experiencia profunda en Airflow comprendan el significado de cada parámetro.



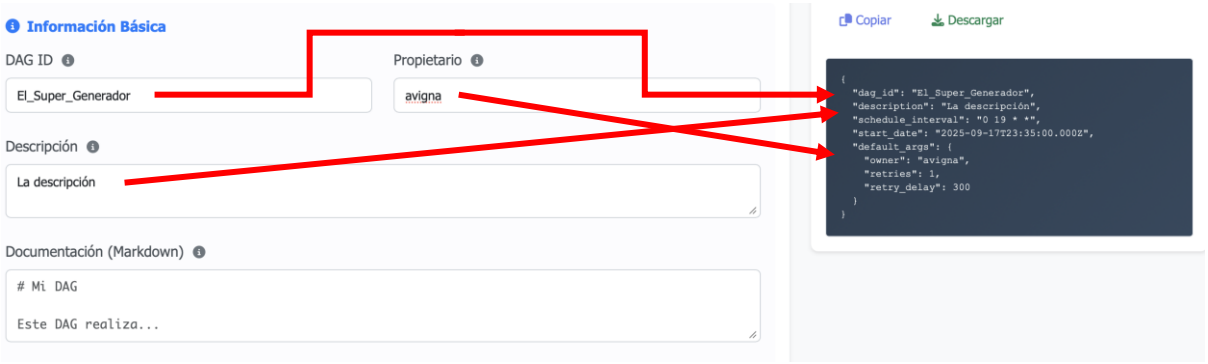
- Validaciones en Tiempo Real:

  El sistema aplica reglas de validación en los campos críticos, como expresiones cron, fechas y direcciones de correo electrónico. De esta manera, se evita la generación de configuraciones erróneas que podrían interrumpir la ejecución del DAG.



- Vista Previa del JSON dinámicamente:

  La vista previa a la derecha del formulario refleja en tiempo real los cambios realizados en la configuración. Esto brinda confianza al usuario, al mostrar cómo será representado el DAG en el archivo final antes de exportarlo.



Secciones de Configuración

La interfaz de configuración está dividida en bloques temáticos que representan los principales parámetros de un DAG en Apache Airflow. A continuación, se describen en detalle los atributos disponibles en las tres primeras secciones: **Información Básica**, **Programación y Fechas**, y **Configuración de Ejecución**.

Información Básica

Esta sección recopila los parámetros iniciales y descriptivos del DAG. Son los atributos que permiten identificarlo y documentarlo dentro del entorno de Airflow.

- ### DAG ID:

  Descripción: Identificador único del DAG dentro de la instalación de Airflow. No puede repetirse, ya que actúa como clave principal para distinguir cada flujo.

  Ejemplo: DAG_ETL_Flujo_Diario

- ### Propietario (Owner):

  Descripción: Usuario responsable del DAG. Generalmente corresponde al desarrollador, equipo o área encargada de su mantenimiento.

  Ejemplo: a-vigna , Equipo_PySpark

- ### Descripción:

  Descripción: Texto breve que explica el propósito del DAG y su función dentro del ecosistema de datos.

  Ejemplo: DAG que procesa y consolida las ventas diarias para el dashboard financiero.

- ### Documentación (Markdown):

  Descripción: Campo opcional que permite añadir documentación extendida en formato Markdown, visible desde la interfaz de Airflow. Útil para agregar detalles técnicos, instrucciones o enlaces de referencia.

  Ejemplo:

  ```
  # DAG ETL _Flujo_Diario

  Este DAG extrae los datos de las transacciones realizadas desde la base de datos, los transforma para que queden según lo solicitado por el cliente y los exporta.

  - Frecuencia: Diaria.
  - Responsable: Equipo_01.
  ```

Programación y Fechas

En esta sección se establecen las reglas de ejecución periódica del DAG y los límites temporales de validez.

- ### Programación (Cron):

  Descripción: Define la periodicidad de ejecución mediante una expresión cron. También se ofrecen botones de selección rápida para configuraciones comunes

  Ejemplo:
  - 0 9 * * * : Todos los días a las 09:00AM.
  - 0 */6 * * * : Cada 6 horas.

- ### Fecha de Inicio (Start Date):

  Descripción: Fecha y hora a partir de la cual el DAG comenzará a ejecutarse. Es obligatoria y debe ser anterior a la fecha actual para que el DAG se active correctamente.

  Ejemplo: 2025-01-05 09:00

- ### Fecha de Fin (End Date):

  Descripción: Fecha límite opcional hasta la cual el DAG podrá ejecutarse. Si se deja vacío, el DAG se considera como indefinido en el tiempo (no va a finalizar).

  Ejemplo: 2025-02-05 23:00.

Configuración de Ejecución

Permite definir límites de concurrencia, control de ejecución paralela y parámetros generales sobre cómo se comportará el DAG durante su operación.

- ### Max Ejecuciones Activas (max_active_runs):

  Descripción: Número máximo de instancias del DAG que pueden ejecutarse en paralelo.

  Ejemplo: 1 (solo una ejecución a la vez, por ejemplo, en un DAG diario y por alguna razón una ejecución aún está corriendo cuando llega la siguiente fecha de ejecución, Airflow **no lanzará la nueva ejecución** hasta que termine la anterior).

- ### Concurrencia DAG (concurrency):

  Descripción: Número máximo de tareas que pueden ejecutarse **en paralelo** dentro del DAG. Si se deja vacío, no hay límite.

  Ejemplo: 5 (máximo cinco tareas simultáneas).

- ### Max Tareas Activas (max_active_tasks):

  Descripción: Número máximo de tareas activas por ejecución del DAG (run).

  Ejemplo: 10.

- ### Timeout DAG Run (dagrun_timeout):

  Descripción: Tiempo máximo (en minutos) que puede durar una ejecución completa del DAG antes de ser marcada como fallida.

  Ejemplo: 120 (dos horas).

- ### Vista Predeterminada (default_view):

  Descripción: Selecciona la vista que se mostrará por defecto en la interfaz gráfica de Airflow al abrir el DAG.

  Opciones: *graph, tree, duration, gantt, landing_times*.

- ### Orientación del Gráfico (orientation):

  Descripción: Define la dirección en la que se dibuja el grafo de tareas.

  Opciones: *LR* (izquierda a derecha), *TB* (arriba hacia abajo), *RL* (derecha a izquierda), *BT (*abajo hacia arriba).

- ### Pausado al Crear (is_paused_upon_creation):

  Descripción: Si está activado, el DAG se crea en estado pausado, evitando ejecuciones automáticas hasta que se active manualmente.

  Ejemplo: true (es decir, al crear el DAG y desplegarlo, este iniciará pausado).

Notificaciones por Email

Esta sección permite configurar alertas automáticas que se enviarán a través de correo electrónico según diferentes eventos en la ejecución del DAG. Es fundamental para la **monitorización** y la **rápida detección de fallos.**

- ### Lista de Correos Electrónicos (email):

  Descripción: Lista de direcciones de correo que recibirán las notificaciones. Se deben separar con comas.

  Ejemplo: admin@gmail.com, data@hotmail.com

- ### Email en Fallo (email_on_failure):

  Descripción: Envía una notificación por correo cuando una tarea falla.

  Ejemplo: true (se envía un correo cada vez que una tarea falla).

- ### Email en Reintento (email_on_retry):

  Descripción: Envía una notificación cuando una tarea entra en reintento tras un fallo.

  Ejemplo: true (notificación al primer reintento de una tarea fallida).

- ### Email en Éxito (email_on_success):

  Descripción: Envía una notificación cuando una tarea finaliza correctamente.

  Ejemplo: false (en ese caso, no enviaría correos en las tareas exitosas).

Reintentos y Timeouts

Esta sección define la política de tolerancia a fallos y los límites de tiempo en la ejecución de tareas. Permite ajustar la **resiliencia** del DAG según la criticidad del proceso.

- ## Número de Reintentos (retries):

  Descripción: Número de veces que se reintentará ejecutar una tarea en caso de fallo.

  Ejemplo: 3 (intenta ejecutar la tarea hasta 3 veces antes de marcarla como fallida).

- ## Retraso entre Reintentos (retry_delay):

  Descripción: Tiempo en segundos que se espera entre un intento fallido y el siguiente

  Ejemplo: 300 (5 minutos entre cada reintento).

- ## Max Retraso Reintento (max_retry_delay):

  Descripción: Tiempo máximo de espera entre reintentos, utilizado en combinación con backoff exponencial.

  Ejemplo: 600 (nunca esperar más de 10 minutos entre reintentos).

- ## Timeout Ejecución (execution_timeout):

  Descripción: Tiempo máximo en segundos permitido para que una tarea individual finalice su ejecución.

  Ejemplo: 1800 (en ese caso, una tarea puede durar hasta 30 minutos).

- ## Timeout General (timeout):

  Descripción: Tiempo máximo global en segundos para todas las tareas, aplicable como un límite adicional.

  Ejemplo: 3600 (ninguna tarea del DAG puede exceder una hora de ejecución total).

- SLA (sla):

  Descripción: Define un Service Level Agreement, expresado en segundos, que establece el tiempo máximo esperado para que una tarea finalice. Si se supera, se genera una alerta de incumplimiento

  Ejemplo: 900 (cada tarea debe terminar en un máximo de 15 minutos).

- Backoff Exponencial en Reintentos (retry_exponential_backoff:

  Descripción: Si está activado, el retraso entre reintentos se incrementa de forma exponencial en lugar de fija, hasta alcanzar el max_retry_delay.

  Ejemplo: true (los reintentos ocurren en intervalos crecientes: 5 minutos, 10 minutis, 20 minutos...).

- Depende del Pasado (depends_on_past):

  Descripción: Si está activado, la ejecución de una tarea depende del éxito de la tarea anterior en el DAG.

  Ejemplo: false (con eso, cada ejecución será independiente).

Configuración Avanzada

Esta sección agrupa parámetros que otorgan mayor control sobre la ejecución del DAG, incluyendo asignación de recursos, reglas de disparo de tareas (triggers) y ajustes de priorización. Son atributos utilizados principalmente en **entornos de producción** donde la optimización de recursos y el control de dependencias son críticos.

- Pools y Queues
- *Pool (pool):*
  - **Descripción:** Nombre del pool al cual se asigna la tarea. Los pools permiten limitar cuántas tareas pueden ejecutarse en paralelo dentro de un mismo grupo de recursos.

- **Ejemplo:** etl_pool → Se utiliza para que todas las tareas de extracción y carga de datos compartan un mismo grupo de recursos, evitando saturar la infraestructura.

- *Pool Slots (pool_slots):*
  - **Descripción:** Número de *slots* que esta tarea consume dentro del pool. Por defecto es 1, pero se puede incrementar si la tarea requiere más recursos.
  - **Ejemplo:** 2 → Se configura cuando una tarea de procesamiento intensivo necesita reservar más capacidad que las demás dentro del mismo pool, garantizando así que disponga de recursos suficientes.

- *Queue (queue):*
  - **Descripción:** Define en qué cola se ejecutará la tarea. Esto permite dirigir tareas específicas a workers designados.
  - **Ejemplo:** gpu_queue → Se asigna a tareas que requieren procesamiento gráfico intensivo, como análisis de imágenes o modelos de machine learning, las cuales deben ejecutarse en nodos con GPU.

- Prioridad y Peso

- *Peso de Prioridad (priority_weight):*
  - **Descripción:** Número entero que representa la prioridad relativa de la tarea al competir por recursos. Tareas con mayor peso tienen mayor prioridad.
  - **Ejemplo:** etl_pool → Se usa para asegurar que una tarea de carga de datos críticos al *data warehouse* se ejecute antes que otras tareas de menor relevancia, como la generación de reportes internos.

- *Regla de Peso (weight_rule):*
  - **Descripción:** Define cómo se calcula el peso de una tarea considerando sus dependencias.
  - **Ejemplo:** downstream → Es útil cuando una tarea de inicio de pipeline afecta a muchas tareas posteriores; al configurarla así, su prioridad se incrementa proporcionalmente a la importancia de su descendencia.

- ## Trigger Rules y Callbacks

- *Trigger Rule (trigger_rule):*
  - **Descripción:** Establece bajo qué condición se ejecuta una tarea en función del estado de sus tareas predecesoras.
  - **Ejemplo:** all_success → Una tarea de consolidación de datos se ejecutará únicamente cuando todas las tareas de extracción previas hayan terminado exitosamente.

- *Callback SLA Miss (sla_miss_callback):*
  - **Descripción:** Función de callback que se ejecuta cuando una tarea incumple un SLA definido.
  - **Ejemplo:** notificar_sla_incumplido → Se implementa para enviar un mensaje al canal de monitoreo si la generación del reporte diario excede el tiempo máximo permitido de 15 minutos.
- *Callback al Fallo (on_failure_callback):*
  - **Descripción:** Función personalizada que se ejecuta automáticamente cuando una tarea falla. Puede usarse para enviar alertas o registrar logs.
  - **Ejemplo:** alerta_fallo_tarea → Se activa para notificar al equipo de soporte mediante correo electrónico cuando una tarea de carga a base de datos falla.

- *Callback en Éxito (on_success_callback):*
  - **Descripción:** Función que se ejecuta cuando una tarea finaliza con éxito. Suele usarse para tareas críticas que deben notificar al completarse.
  - **Ejemplo:** notificar_exito → Se implementa para enviar un mensaje de confirmación al sistema de monitoreo cada vez que se completa el proceso de facturación mensual.

- *Callback en Reintento (on_retry_callback):*
  - **Descripción:** Función que se dispara cuando una tarea entra en reintento tras un fallo.
  - **Ejemplo:** registrar_reintento → Se utiliza para guardar un registro en una tabla de auditoría cada vez que una tarea de extracción de datos falla y necesita volver a intentarse.

- Otras Configuraciones
- *Tags (tags):*
  - **Descripción:** Lista de etiquetas que permiten clasificar y filtrar DAGs en la interfaz de Airflow.
  - **Ejemplo:** ETL_001_PROD → Se asigna para identificar que el DAG pertenece al flujo de extracción de datos y que se ejecuta en ambiente productivo.

- *Catchup (catchup):*
  - **Descripción:** Define si el DAG debe ejecutar instancias pasadas que no se corrieron cuando estuvo inactivo.
  - **Ejemplo:** false → e utiliza en procesos diarios en tiempo real, como el monitoreo de servidores, donde no tiene sentido ejecutar ejecuciones atrasadas.
- *Provide Context (provide_context):*
  - **Descripción:** Si está activado, las funciones Python reciben automáticamente el contexto de ejecución (fechas, run_id, etc.) como parámetros.
  - **Ejemplo:** true→ Se activa en una tarea Python que necesita conocer la fecha de ejecución para solo realizar la extracción de ese día.

Editor Dinámico de JSON:

El Editor Dinámico de JSON es una de las funcionalidades más flexibles del generador, ya que permite crear, modificar y extender objetos en formato JSON directamente desde la interfaz gráfica. Esta característica está pensada para escenarios donde el DAG requiere configuraciones personalizadas que no están cubiertas por los campos estándar del formulario. El editor brinda la posibilidad de definir estructuras anidadas, listas y pares clave-valor, garantizando que el resultado siempre cumpla con el formato correcto de JSON.

Su uso es especialmente útil en contextos donde se necesita añadir parámetros específicos de negocio, conexiones externas o configuraciones propias de cada organización. El editor cuenta con controles para agregar o eliminar atributos y objetos, lo que facilita la experimentación y la personalización. De esta forma, los equipos técnicos pueden enriquecer la definición del DAG sin tener que modificar

manualmente el código Python en Airflow, manteniendo la estandarización y reduciendo riesgos de errores de sintaxis.